

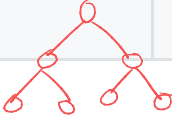
问题复杂度

概念

- 1. 原地工作的算法
- 2. 最坏情况下最优和平均情况下最优
- 3. 问题的时间复杂性的上界与下界
- 4. 问题的时间复杂度

检索问题时间复杂度: $\lfloor \log n \rfloor + 1$

算法	最坏情况	平均情况	占用空间	时间最优性
顺序检索	n	$\frac{3n}{4}$	原地	
二分检索	$\lfloor \log n \rfloor + 1$	$\lfloor \log n \rfloor + \frac{1}{2}$	原地	最坏情况最优



排序问题时间复杂度: $\Theta(n \log n)$

算法	最坏情况	平均情况	占用空间	时间最优性
冒泡排序	$\frac{n(n-1)}{2} = O(n^2)$	$\frac{n(n-1)}{4} = O(n^2)$	原地	
快速排序	$O(n^2)$	$O(n \log n)$	$O(\log n)$	平均情况最优
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n)$	最优
堆排序	$n \log n - 1.5n = O(n \log n)$	$O(n \log n)$	原地	最优



选择问题时间复杂度

问题	算法	最坏情况	问题复杂度	时间最优性
找最大	FindMax	$n - 1$	$n - 1$	最优
找最大最小	FindMaxMin	$\lceil \frac{3n}{2} \rceil - 2$	$\lceil \frac{3n}{2} \rceil - 2$	最优
找第二大	锦标赛	$n + \lceil \log n \rceil - 2$	$n + \lceil \log n \rceil - 2$	最优
找中位数	Select	$O(n)$	$\frac{3n}{2} - \frac{3}{2}$	阶最优
找第k小	Select	$O(n)$	$n + \min\{k, n - k + 1\} - 2$	阶最优

问题复杂度

概念

- 1. 原地工作的算法
- 2. 最坏情况下最优和平均情况下最优
- 3. 问题的时间复杂性的上界与下界
- 4. 问题的时间复杂度

检索问题时间复杂度: $\log n + 1$

算法	最坏情况	平均情况	占用空间	时间最优性
顺序检索	n	$\frac{3n}{4}$	原地	
二分检索	$\lceil \log n \rceil + 1$	$\lceil \log n \rceil + \frac{1}{2}$	原地	最坏情况最优

排序问题时间复杂度: $\Theta(n \log n)$

算法	最坏情况	平均情况	占用空间	时间最优性
冒泡排序	$\frac{n(n-1)}{2} = O(n^2)$	$\frac{n(n-1)}{4} = O(n^2)$	原地	
快速排序	$O(n^2)$	$O(n \log n)$	$O(\log n)$	平均情况最优
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n)$	最优
堆排序	$n \log n - 1.5n = O(n \log n)$	$O(n \log n)$	原地	最优

选择问题时间复杂度

问题	算法	最坏情况	问题复杂度	时间最优性
找最大	FindMax	$n - 1$	$n - 1$	最优
找最大最小	FindMaxMin	$\lceil \frac{3n}{2} \rceil - 2$	$\lceil \frac{3n}{2} \rceil - 2$	最优
找第二大	锦标赛	$n + \lceil \log n \rceil - 2$	$n + \lceil \log n \rceil - 2$	最优
找中位数	Select	$O(n)$	$\frac{3n}{2} - \frac{3}{2}$	阶最优
找第k小	Select	$O(n)$	$n + \min\{k, n - k + 1\} - 2$	阶最优

- 8.1 设 $G_1 = \langle V_1, E_1 \rangle, G_2 = \langle V_2, E_2 \rangle$ 是两个简单图, 其中 $V_1 = V_2$. 假设 G_1 和 G_2 的输入是用邻接矩阵表示的. 换句话说, 如果 (v_i, v_j) 是图的边, 那么它的邻接矩阵 M 的第 i 行第 j 列的元素 $r_{ij} = 1$; 否则 $r_{ij} = 0, 1 \leq i, j \leq n$.
- (1) 设输入规模是图中的顶点数 n , 给出一个算法判定 G_1 是否为 G_2 的补图. 说明算法的设计思想, 并给出最坏情况下的时间复杂度.
- (2) 对于求解这个问题的所有算法, 给出一个尽可能紧的时间复杂度的下界, 并证明你的结果.

解: (1) 逐个比较 G_1 和 G_2 中的元素, 若 $\exists (i, j), G_1[i, j] = G_2[i, j]$ 则 G_1 不是 G_2 的补图, 否则 G_1 是 G_2 的补图. 由于 G_1, G_2 是无向图, 故只需比较上三角矩阵. $W(n) = \frac{n(n-1)}{2}$

(2) 对于求解该问题的任意算法 A , A 在最坏情况下的时间复杂度下界为 $\frac{n(n-1)}{2}$ 对称

证: 该问题等价于比较两个大小为 $n \times n$ 的 0-1 矩阵是否互补, 要求解该问题, 最坏情况下需进行至少 $\frac{n(n-1)}{2}$ 次比较以检查所有元素

$$\text{故 } W(n) = \frac{n(n-1)}{2}$$

8.2 对于给定的 $x \neq 0$, 求 n 次多项式 $P(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$ 的值.

(1) 设计一个在最坏情况下时间复杂度为 $\Theta(n)$ 的求值算法.

(2) 证明任何求值算法的时间复杂度都是 $\Omega(n)$

解: (1) 采用分治算法.

$$P(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

$$= P_{\text{even}}(x) + xP_{\text{odd}}(x)$$

① n 为偶数

$$P_{\text{even}}(x) = a_0 + a_2x + \dots + a_nx^{\frac{n}{2}}$$

$$P_{\text{odd}}(x) = a_1 + a_3x + \dots + a_{n-1}x^{\frac{n}{2}-1}$$

② n 为奇数

$$P_{\text{even}}(x) = a_0 + a_2x + \dots + a_{n-1}x^{\frac{n-1}{2}}$$

$$P_{\text{odd}}(x) = a_1 + a_3x + \dots + a_nx^{\frac{n-1}{2}}$$

$$\Rightarrow T(n) = 2T(\frac{n}{2}) + O(1)$$

$$\Rightarrow T(n) = \Theta(n)$$

(2) 证: $\forall$ 求值算法 A .

$$P(x) \text{ 是 } n \text{ 次多项式} \Rightarrow a_n \neq 0$$

$$A \text{ 求 } P(x=x_0) \text{ 至少要做 } n-1+1=n \text{ 次运算 } (x_0 \neq 0)$$

$$\Rightarrow A \text{ 的时间复杂度 } T(n) = \Omega(n)$$

(以乘法为关键运算).

8.2 (1) 迭代计算:

$$P_1(x) = a_n$$

$$P_2(x) = a_{n-1} + P_1(x) \cdot x$$

$$P_3(x) = a_{n-2} + P_2(x) \cdot x$$

$$\vdots$$

$$P_{n+1}(x) = a_0 + P_n(x) \cdot x$$

不难看出

$$P_{n+1}(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n = P(x)$$

(2) 证 多项式 $P(x)$ 有 $n+1$ 个数, 每个系数至少需要被处理 1 次. 如若不然, 假设某个求值算法 A 不对系数 a_i 进行处理. 考虑输入

$$\langle a_0, a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n \rangle \text{ 和 } \langle a_0, a_1, \dots, a_{i-1}, b_i, a_{i+1}, \dots, a_n \rangle$$

其中 $a_i \neq b_i$ 是不为 0 的数, 其他系数都相等. 那么这两个多项式对同一个 x 的求值一定不等, 但是算法 A 的输出是不加区别的. 因此算法 A 出错. 于是任何算法至少需要 $n+1$ 次运算, 即在最坏情况下时间复杂度为 $\Omega(n)$.

8.3 (1) 设 A 和 B 是两个长为 n 的有序数组, 现在需要将 A 与 B 合并成一个排好序的数组; 证明任何以元素比较作为基本运算的归并算法至少要做 $2n-1$ 次比较.

(2) 对上述归并问题, 假设 $|A|=m, |B|=n$, 给出求解该问题的最优算法并证明其最优性.

解: (1) 构造坏输入.

$$\text{设 } A = [a_1, a_2, \dots, a_n], B = [b_1, b_2, \dots, b_n]$$

$$\text{且 } a_1 < b_1 < a_2 < b_2 < \dots < b_{i-1} < a_i < b_i < \dots < a_n < b_n$$

对 $\forall a_i, i \geq 1$, a_i 至少与 b_{i-1} 和 b_i 比较后才能确定自己的位置. a_i 只需与 b_i 比较 1 次即可.

因此对任意算法, 至少需进行 $2(n-1) + 1 = 2n-1$ 次比较.

(2) 不妨设 $n \geq m$.

$$\textcircled{1} m = \theta(n)$$

$$\text{常规的归并, } W(n) = O(m+n-1)$$

$$\textcircled{2} m = \theta(\log n)$$

二分归并, $W(n) = \theta(m \log n) = \theta(\log^2 n)$

最优性证明: 构建决策树

$$\text{树叶数: } C_{n+m}^m = \frac{(m+n)!}{m!n!} \quad d = \lceil \log t \rceil$$

$$\text{树叶 } d = \lceil \log C_{n+m}^m \rceil \geq \log(m+n)! - \log m! - \log n!$$

$$\geq (m+n) \log(m+n) - m \log m - n \log n$$

$$\textcircled{1} m = \theta(n) = cn$$

$$\Rightarrow d \geq (c+1)n \log(c+1)n - cn \log cn - n \log n$$

$$= cn \log(c+1)n - cn \log cn + n \log(c+1)n - n \log n$$

$$= cn \log \frac{c+1}{c} + n \log(c+1)$$

$$= m \log \frac{c+1}{c} + n \log(c+1)$$

$$\textcircled{2} m = \theta(\log n) = c \log n$$

$$\Rightarrow d \geq (c \log n + n) \log(c \log n + n) - c \log n \log c \log n - n \log n$$

$$= c \log n \log \frac{c \log n + n}{c \log n} + n \log \frac{c \log n + n}{n}$$

$$= m \log(1 + \frac{n}{m}) + n \log(1 + \frac{m}{n})$$

$$> m \log(1 + \frac{n}{m}) > m \log \frac{n}{m} = \theta(m \log n) = \theta(\log^2 n)$$

$$\lceil \frac{2n}{2} \rceil = n$$

8.4 设 n 是 k 的倍数, 有 k 个排好序的数表 $L_1, L_2, \dots, L_k$, 每个数表都有 n/k 个数. 假设 n 个数彼此不等, 并且归并长为 m, n 的两个数表的时间代价是 $O(m+n)$.

(1) 使用顺序归并算法归并这 k 个数表, 在最坏情况下的时间复杂度是什么?

(2) 设计一个时间复杂度更低的归并算法, 说明算法的主要设计思想, 并分析你的算法在最坏情况下的时间复杂度.

(3) 对于以比较作为基本运算求解上述问题的算法类, 最坏情况下的时间复杂度的下界是什么? 证明你的结果.

解: (1) 最坏情况下归并长为 m, n 的两个数表的时间代价为 $m+n-1$.

故算法在最坏情况下的时间复杂度为

$$\frac{2n}{k} + \frac{3n}{k} + \frac{4n}{k} + \dots + \frac{kn}{k} = \frac{n}{k} \cdot \frac{(k+2)(k-1)}{2} = \theta(k \cdot n)$$

(2) 采用分治策略, 每次将数表均分为两部分, 分别归并两部分后再合并起来.

$$W(k, n) = 2W\left(\frac{k}{2}, \frac{n}{2}\right) + O(n) \quad \text{递归树}$$

$$\Rightarrow W(k, n) = \theta(n \log k)$$

(3) 下界为 $\theta(n \log k)$, 证明如下:

用决策树, 设 n 个数为 $a_1, a_2, \dots, a_n$, 构造规则如下:

设当前结点比较 i, j , 记为 (i, j)

① $i < j$,

若算法下一步比较 i', j' , 则将 (i, j) 的左子结点标记为 (i', j') ; 若算法结束, 则将 (i, j) 的左子结点标记为叶子结点.

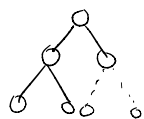
② $i > j$

若算法下一步比较 i', j' , 则将 (i, j) 的右子结点标记为 (i', j') ; 若算法结束, 则将 (i, j) 的右子结点标记为叶子结点.

每个叶子结点对应一种输入.

$$\text{所有可能的输入有 } \frac{n!}{\underbrace{\left(\frac{n}{k}\right)! \left(\frac{n}{k}\right)! \dots \left(\frac{n}{k}\right)!}_{k \uparrow}} = \frac{n!}{\left(\left(\frac{n}{k}\right)!\right)^k} \text{ 种}$$

$$\Rightarrow \text{树深 } d \geq \left\lceil \log \frac{n!}{\left(\left(\frac{n}{k}\right)!\right)^k} \right\rceil$$



$$\geq \log n! - k \log \left(\frac{n}{k}\right)!$$

$$= \theta(\log n! - k \cdot \frac{n}{k} \log \frac{n}{k})$$

$$= \theta(n \log n - n \log n + n \log k)$$

$$= \theta(n \log k)$$

$\Rightarrow$ 下界为 $\theta(n \log k)$